# Backend + AI Engineering Take-Home Assignment

## Overview

This assignment is designed to evaluate how you think, structure systems, debug problems, make trade-offs, and use AI/tools effectively in real-world engineering work.

You may use:

- AI tools (ChatGPT, Claude, Copilot, Cursor, etc.)
- Online resources
- Any libraries/frameworks

What matters is:

- Your reasoning
- Your implementation quality
- Your decisions
- Your ability to ship something reliable and thoughtful

---

# Time Expectation

You have 48 hours from the time you receive this assignment.

We do not expect a “perfect production system”.
We expect a strong demonstration of:

- engineering judgment
- ownership
- problem-solving
- AI-assisted workflow usage

---

# Assignment: Intelligent Media Processing Pipeline

Build a backend system that accepts uploaded images and processes them asynchronously.

The system should:

1. Accept image uploads through an API
2. Store metadata
3. Process images asynchronously
4. Detect possible issues in the uploaded image
5. Provide processing status APIs
6. Generate structured analysis results

---

# Problem Context

Imagine users are uploading vehicle images from the field.

The uploaded image may have issues such as:

- blurry image
- low light
- duplicate image
- screenshot/photo-of-photo
- edited/tampered-looking image
- invalid vehicle number format

Your system should attempt to identify and report these issues.

The goal is NOT perfect ML accuracy.
The goal is to evaluate:

- system design
- reasoning
- engineering quality
- ability to structure uncertainty

---

# Technical Requirements

## Core Requirements

### 1. Upload API

Create an API to upload an image.

Requirements:

- Accept image upload
- Generate unique ID
- Save image locally or cloud storage
- Store metadata in database
- Return processing ID immediately

---

### 2. Async Processing

Image analysis MUST happen asynchronously.

Requirements:

- Queue-based or background-job architecture
- Processing status states:
    - pending
    - processing
    - completed
    - failed

You may use:

- BullMQ
- SQS
- RabbitMQ
- in-memory queue
- custom implementation

Choice matters less than reasoning.

---

### 3. Image Analysis

Implement at least 4 meaningful checks.

Examples:

- blur detection
- brightness analysis
- duplicate detection
- OCR extraction
- Indian number plate format validation
- image dimension validation
- screenshot detection heuristics
- photo-of-photo heuristics
- metadata analysis
- suspicious editing heuristics

You may use:

- OpenCV
- AWS Rekognition
- OCR libraries
- AI APIs
- custom heuristics

You are encouraged to combine multiple approaches.

---

## 4. Results API

Create APIs to:

- fetch processing status
- fetch analysis results
- fetch failure reason (if failed)

---

## 5. Persistence

Use a database.

Recommended:

- PostgreSQL
- MySQL
- MongoDB

Your schema design should be reasonable.

---

# Required Deliverables

## 1. Source Code

Provide complete source code.

---

## 2. README (Important)

Your README should include:

### Architecture

Explain:

- service flow
- processing flow
- queue strategy
- major design decisions

---

### AI Usage Disclosure (Mandatory)

Describe:

- where you used AI
- what AI helped with
- where AI output was wrong
- how you validated AI-generated code

We strongly value thoughtful AI usage.

---

### Trade-offs

Explain:

- what you intentionally simplified
- what you would improve with more time
- scalability concerns
- failure handling concerns

---

### Running Instructions

Should be easy to run locally.

Bonus points for:

- Docker setup
- seed scripts
- test scripts

---

# Evaluation Criteria

## 1. Engineering Quality

We evaluate:

- code structure
- readability
- maintainability
- API design
- modularity

---

## 2. Problem Solving

We evaluate:

- how you approached ambiguity
- how you reasoned about heuristics
- how you handled uncertainty

---

## 3. System Thinking

We evaluate:

- async design
- failure handling
- retries
- scalability awareness
- data modeling

---

## 4. Debugging & Reliability Mindset

We evaluate:

- logging
- error handling
- edge cases
- resilience

---

## 5. AI-Assisted Workflow Maturity

We evaluate:

- how effectively you used AI
- whether you validated outputs
- whether you used AI strategically vs blindly

---

# Bonus Points (Optional)

These are NOT mandatory.

Possible bonus areas:

- dashboard/UI
- analytics
- confidence scoring

- retry mechanisms
- concurrency handling
- automated tests
- Docker Compose
- observability/logging improvements
- rate limiting
- deployment
- cost optimization thinking
- benchmark/performance analysis

---

# Important Notes

We care more about:

- thoughtful engineering
- reasoning
- ownership
- clarity

than about:

- flashy frameworks
- overengineering
- perfect UI

A smaller but well-thought-out solution is better than a huge incomplete one.

---

# Submission

Please submit:

1. Git repository link
2. README
3. Any setup instructions
4. Sample API requests/responses
5. Any assumptions you made

---

# What We’re Looking For

Strong candidates usually:

- make reasonable trade-offs
- communicate clearly
- think about edge cases
- structure code well
- use AI intelligently
- demonstrate ownership
- build systems that are practical and debuggable